INSTITUTE OF ENGINEERING & MANAGEMENT

SECTOR V, KOLKATA

PCCCS495: Advanced Programming (OOP) Lab

TOPIC : Smart Study Planner using Strategy and Observer Pattern

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (CSE - IOTCSBT),

2025-2026

Instructor name: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Name : Debjit Dey	**Class Roll :** 02
Course : B.Tech	**Stream :** CSE – [IOTCSBT]
Semester : 4th	**Enrollment No.:** 12024052017019

Problem Statement

Students often face difficulty in managing multiple academic tasks such as assignments, exams, and study activities due to the lack of structured planning and prioritization. Tasks are typically handled in an unorganized manner, leading to missed deadlines, inefficient time allocation, and increased stress.

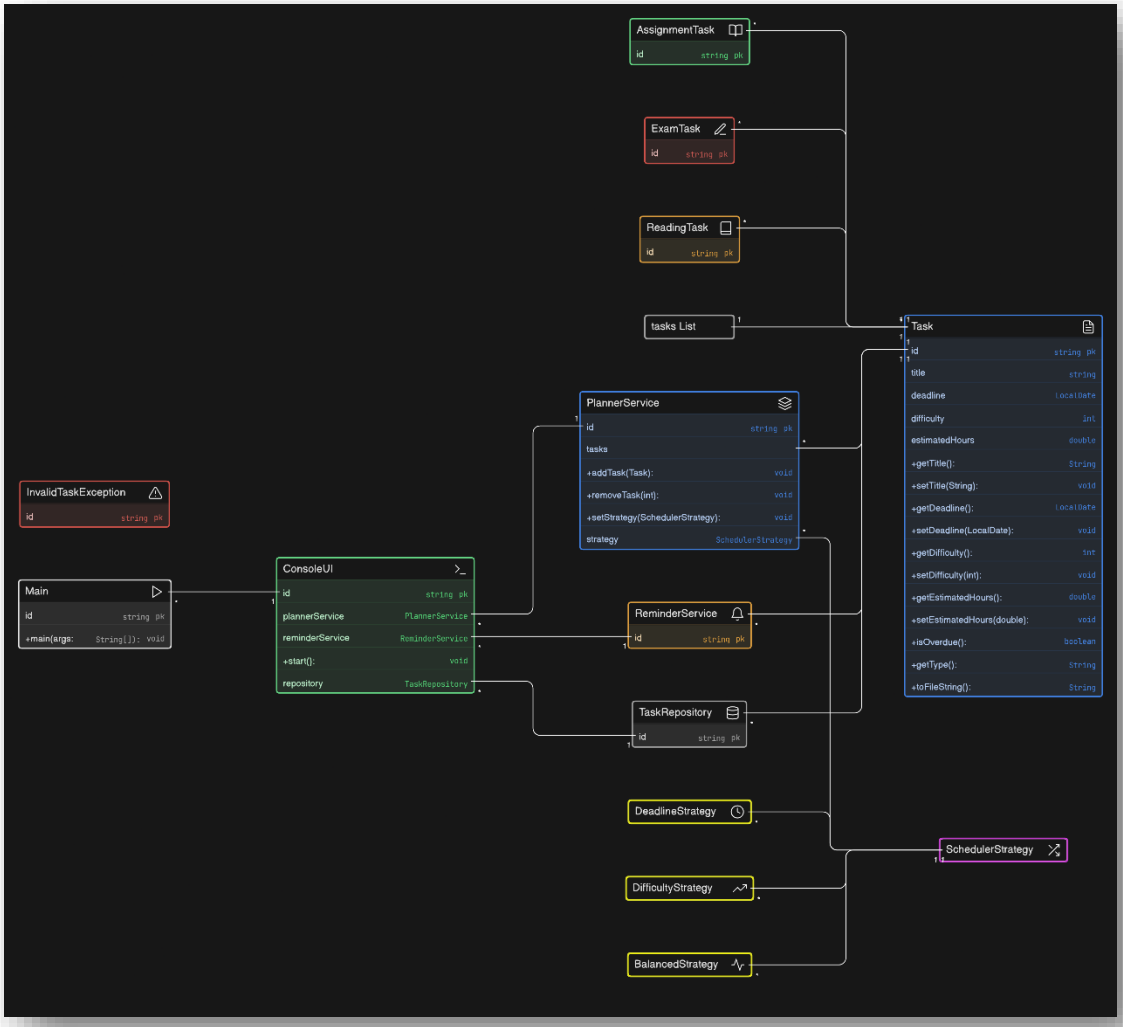
Existing simple planners or to-do list systems generally follow a fixed prioritization approach and do not adapt to different user needs. However, in real scenarios, task prioritization can vary depending on factors such as deadlines, difficulty level, and required effort.

To address this issue, there is a need for a flexible and structured system that allows dynamic prioritization of tasks based on different strategies. The system should enable users to manage tasks efficiently, switch between different scheduling methods, and receive timely alerts for upcoming or overdue tasks.

The proposed solution is a Smart Study Planner that applies object-oriented design principles and design patterns to provide a modular, extensible, and user-friendly task management system.

System Design

UML Diagram :



Major Classes

The system is composed of multiple classes organized into different layers, each responsible for a specific functionality.

- **Task (Abstract Class)**
  Represents the base structure for all tasks. It defines common attributes such as title, deadline, difficulty, and estimated hours, along with shared behaviors.
- **AssignmentTask, ExamTask, ReadingTask**
  These classes extend the Task class and represent specific types of academic tasks.
- **SchedulerStrategy (Interface)**
  Defines a common method for scheduling tasks. It enables different scheduling approaches to be implemented.

- **DeadlineStrategy, DifficultyStrategy, BalancedStrategy**
  These classes implement the SchedulerStrategy interface and provide different task prioritization logics.
- **PlannerService**
  Acts as the core logic handler. It manages task operations such as adding, removing, and retrieving tasks, and applies the selected scheduling strategy.
- **ReminderService**
  Responsible for checking deadlines and notifying the user about overdue or upcoming tasks.
- **TaskRepository**
  Handles file-based persistence by saving and loading tasks from a storage file.
- **ConsoleUI**
  Provides a menu-driven interface for user interaction and connects user input to the system logic.
- **InvalidTaskException**
  A custom exception used to handle invalid input scenarios.
- **Main**
  Entry point of the application that initializes components and starts the system.

Package Structure

The project is organized into multiple packages to ensure modularity and separation of concerns:

- **model**
  Contains all task-related classes including the abstract Task class and its subclasses.
- **strategy**
  Contains the scheduling interface and its implementations.
- **service**
  Contains the core business logic classes such as PlannerService and ReminderService.
- **repository**
  Handles data persistence and file operations.
- **ui**
  Manages user interaction through the console interface.
- **exception**
  Contains custom exception classes.

Interaction Flow

The system follows a layered interaction flow where each component communicates with specific layers:

1. The user interacts with the system through the ConsoleUI by selecting options from the menu.
2. The ConsoleUI sends user requests to the PlannerService.
3. The PlannerService performs operations such as adding, removing, or retrieving tasks.
4. When scheduling is required, the PlannerService delegates the operation to the selected SchedulerStrategy.
5. The strategy processes the list of tasks and returns a prioritized result.
6. The ReminderService checks deadlines and provides alerts when required.
7. The TaskRepository is used to save and load tasks from the file system.
8. The Main class initializes all components and starts the interaction.

OOP Concept Used

1. Abstraction

```
Task.java
C: > Users > Debjit > Downloads > term-ii-project-submission-DeeeeeDey > src > SmartStudyPlanner > src > planner > model > Task.java
public abstract class Task {

    private String title;
    private LocalDate deadline;
    private int difficulty;
    private double estimatedHours;

    public Task(String title, LocalDate deadline, int difficulty, double estimatedHours) {
        this.title = title;
        this.deadline = deadline;
        this.difficulty = difficulty;
        this.estimatedHours = estimatedHours;
    }

    public String getTitle() {
        return title;
    }

    public LocalDate getDeadline() {
        return deadline;
    }
```

This snippet demonstrates abstraction through the use of an abstract class Task, which defines common attributes and an abstract method *getType()*. The implementation details of specific task types are deferred to subclasses, allowing a generalized representation of tasks while hiding specific behaviors.

2. Inheritance

```
C: > Users > Debjit > Downloads > term-ii-project-submission-DeeeeeDey > src > SmartStudyPlanner > src > planner > model > Assignment
package planner.model;

import java.time.LocalDate;

public class AssignmentTask extends Task {

    public AssignmentTask(String title, LocalDate deadline, int difficulty, double estimatedHours) {
        super(title, deadline, difficulty, estimatedHours);
    }

    public String getType() {
        return "Assignment";
    }
}
```

This snippet shows inheritance where AssignmentTask extends the base class Task. It reuses common properties and behavior from the parent class while providing its own specific implementation of the getType() method, demonstrating code reuse and specialization.

3. POLYMORPHISM

```java
public class PlannerService {

    private List<Task> tasks;
    private SchedulerStrategy strategy;

    public PlannerService() {
        tasks = new ArrayList<Task>();
    }

    public void setStrategy(SchedulerStrategy strategy) {
        this.strategy = strategy;
    }

    public void addTask(Task task) {
        tasks.add(task);
    }
```

This snippet demonstrates runtime polymorphism using the SchedulerStrategy interface. The PlannerService holds a reference to the strategy and dynamically invokes the schedule() method based on the selected implementation, allowing the system to change behavior at runtime without modifying existing code.

4. ENCAPSULATION

```java
public void setTitle(String title) {
    this.title = title;
}

public void setDeadline(LocalDate deadline) {
    this.deadline = deadline;
}

public void setDifficulty(int difficulty) {
    this.difficulty = difficulty;
}
```

This snippet illustrates encapsulation by keeping the title attribute private and providing public getter and setter methods. This ensures controlled access to the data and protects it from direct modification, maintaining data integrity.

5. Exception Handling

```java
try {
    System.out.print("Difficulty (1-5): ");
    difficulty = Integer.parseInt(sc.nextLine());
} catch (Exception e) {
    System.out.println("Invalid number");
    return;
}
```

This snippet demonstrates exception handling using a try-catch block to safely parse user input. It prevents the program from crashing due to invalid input and provides a user-friendly error message, improving system robustness.

Implementation Highlights

Dynamic Strategy Selection :

```java
public class PlannerService {

    private List<Task> tasks;
    private SchedulerStrategy strategy;

    public PlannerService() {
        tasks = new ArrayList<Task>();
    }

    public void setStrategy(SchedulerStrategy strategy) {
        this.strategy = strategy;
    }

    public void addTask(Task task) {
        tasks.add(task);
    }

    public boolean removeTask(int index) {
        if (index >= 0 && index < tasks.size()) {
            tasks.remove(index);
            return true;
        }
        return false;
    }
```

This snippet represents the core design decision of the system. It allows dynamic selection and application of different scheduling strategies at runtime, making the system flexible and extensible.

Balanced Strategy Scoring :

```java
    private double calculateScore(Task task) {

        long daysLeft = ChronoUnit.DAYS.between(LocalDate.now(), task.getDeadline());

        if (daysLeft <= 0) {
            daysLeft = 1;
        }

        return (task.getDifficulty() * 2.0) + task.getEstimatedHours() + (10.0 / daysLeft);
    }
}
```

This highlights the implementation of a custom scheduling algorithm that combines multiple factors such as difficulty, estimated hours, and deadline. It demonstrates

how domain-specific logic is used to prioritize tasks effectively.

File Persistence Logic :

```
34      public List<Task> loadTasks() {
35
36          List<Task> tasks = new ArrayList<Task>();
37
38          File file = new File(filePath);
39
40  >       if (!file.exists()) {···
42          }
43
44  >       try {···
86  >       } catch (Exception e) {···
88          }
89
90          return tasks;
91      }
92  }
```

This snippet demonstrates file-based persistence, where task data is stored and reconstructed from a file. It shows how object data is converted to a storable format and later recreated, ensuring data continuity across program executions.

Input Handling with Exception Safety :

```
            try {
                choice = Integer.parseInt(sc.nextLine());
>           } catch (Exception e) {···
            }

>           try {···
>           } catch (InvalidTaskException e) {···
            }
        }
    }
```

This snippet shows robust input handling using exception handling techniques. It ensures that invalid user inputs do not crash the system, thereby improving reliability and user experience.

Testing & Error Handling

Test Cases Considered

The system was tested using multiple functional test cases to ensure correct behavior of all features:

- **Task Creation:** Verified that tasks of different types (Assignment, Exam, Reading) are correctly created and stored.
- **Task Viewing:** Checked that all tasks are displayed correctly with complete details.
- **Task Scheduling:** Tested different scheduling strategies (Deadline, Difficulty, Balanced) to ensure tasks are ordered appropriately.
- **Strategy Switching:** Verified that changing the scheduling strategy dynamically updates the task order.
- **Task Removal:** Ensured tasks can be removed using valid indices.
- **Data Persistence:** Tested saving tasks to file and reloading them to confirm data consistency across executions.
- **Reminder Functionality:** Verified that overdue and near-deadline tasks are correctly identified and displayed.

Edge Cases Handled

Several edge cases were considered to ensure robustness of the system:

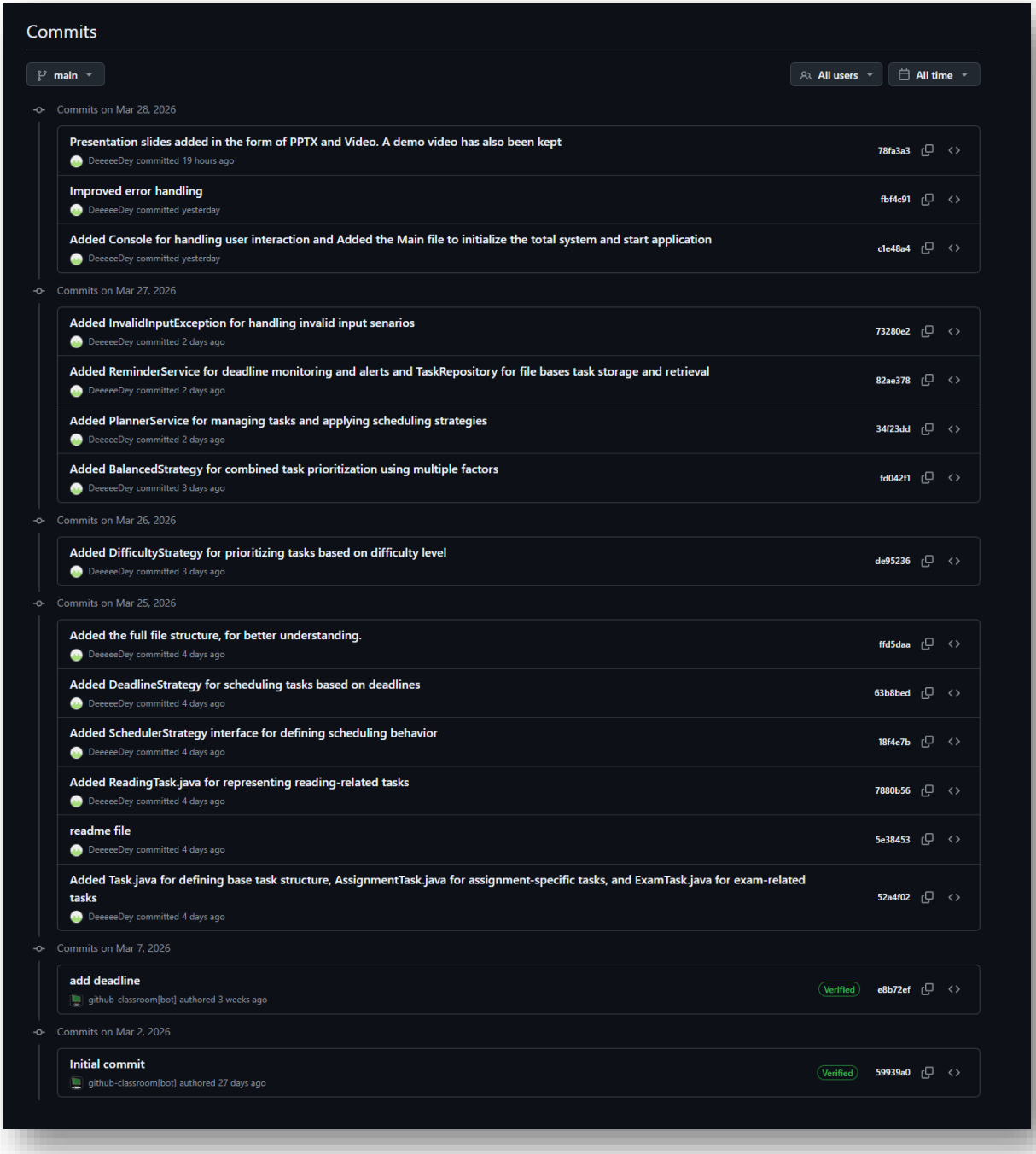
- **Empty Task List:** Viewing or scheduling tasks when no tasks are present does not cause errors.
- **Past Deadlines:** Tasks with deadlines in the past are correctly marked as overdue.
- **Near Deadlines:** Tasks with deadlines within a short time frame are identified as “due soon.”
- **Same Attribute Values:** Tasks with identical deadlines or difficulty levels are handled correctly during sorting.
- **Boundary Values:** Difficulty values at the limits (1 and 5) are processed correctly.
- **Invalid Index Removal:** Attempting to remove a task with an invalid index does not crash the system.

Failure Scenarios

The system includes error handling to manage invalid inputs and unexpected conditions:

- **Invalid Numeric Input:** Non-numeric input for menu options, difficulty, or estimated hours is handled using exception handling without crashing the program.
- **Invalid Date Format:** Incorrect date formats are detected and appropriate error messages are displayed.
- **Empty Title Input:** Tasks with empty titles are rejected using validation checks.
- **File Not Found:** If the data file does not exist, the system initializes with an empty task list instead of failing.
- **Corrupted File Data:** Invalid or improperly formatted lines in the data file are safely ignored during loading.
- **General Runtime Errors:** Try-catch blocks are used to prevent abrupt termination and ensure smooth execution.

GIT WORKFLOW



The project was developed in a step-by-step manner using version control. Initially, the core task model was implemented using an abstract Task class and its subclasses. Next, the scheduling logic was developed using the Strategy Pattern with multiple implementations.

The service layer was then introduced to manage tasks, followed by the addition of file handling and reminder functionalities. Error handling was implemented using custom exceptions and validation. Finally, the user interface and main execution flow were completed, and improvements were made to ensure robustness.

Conclusion

The Smart Study Planner was successfully developed to provide an efficient and flexible solution for managing academic tasks. The system applies object-oriented principles along with the Strategy Pattern to enable dynamic task scheduling based on different criteria such as deadlines, difficulty, and estimated effort.

The modular design ensures clear separation between user interaction, business logic, and data handling. Key features including task management, scheduling, file persistence, and reminder alerts were implemented effectively. Additionally, proper validation and exception handling were incorporated to ensure robustness and reliability.

Future Scope

The Smart Study Planner was successfully developed to provide an efficient and flexible solution for managing academic tasks. The system applies object-oriented principles along with the Strategy Pattern to enable dynamic task scheduling based on different criteria such as deadlines, difficulty, and estimated effort.

The modular design ensures clear separation between user interaction, business logic, and data handling. Key features including task management, scheduling, file persistence, and reminder alerts were implemented effectively. Additionally, proper validation and exception handling were incorporated to ensure robustness and reliability.

Appendix

Initial Project Proposal :

PROJECT PROPOSAL

DEBJIT DEY : IOTCSBT : 02 : 12024052017019

PROJECT TITLE : Smart Study Planner using Strategy and Observer Pattern

PROBLEM STATEMENT :
Students frequently experience issues in managing school work due to improper prioritization, shifting deadlines, and the absence of a structured planning tool. Conventional paper-based planners are static and do not respond to shifting priorities based on the urgency, complexity, or time available for a given task. The project aims to create a Smart Study Planner using Java that automatically organizes academic tasks based on user-defined criteria such as deadlines, difficulty level, and time required for a task. The application also includes features such as multiple scheduling methods, the ability to change methods as required, reminders for approaching deadlines, and notifications for overdue tasks. The project seeks to create a modular, object-oriented solution that reflects the principles of abstraction, extensibility, separation of concerns, and the use of design patterns in a simple and maintainable fashion.

TARGET USER :

College students who want a structured and flexible way to organize academic tasks and manage deadlines.

CORE FEATURES :

- Add, update, and delete academic tasks
- Task categorization (assignment, exam preparation, reading)
- Dynamic scheduling using selectable strategies
- Runtime switching between scheduling strategies
- Deadline reminders and overdue alerts
- File-based persistence for saving and loading tasks
- Input validation and exception handling

OOP CONCEPTS TO BE USED

ABSTRACTION : Will represent shared attributes such as title, deadline, difficulty, and estimated effort while allowing flexibility for specialized task types.

INHERITANCE : The design will include specialized task categories such as assignments, examinations, and reading activities, each extending a generalized task structure to support code reuse and logical hierarchy.

POLYMORPHISM : Different scheduling behaviors dynamically according to user request.

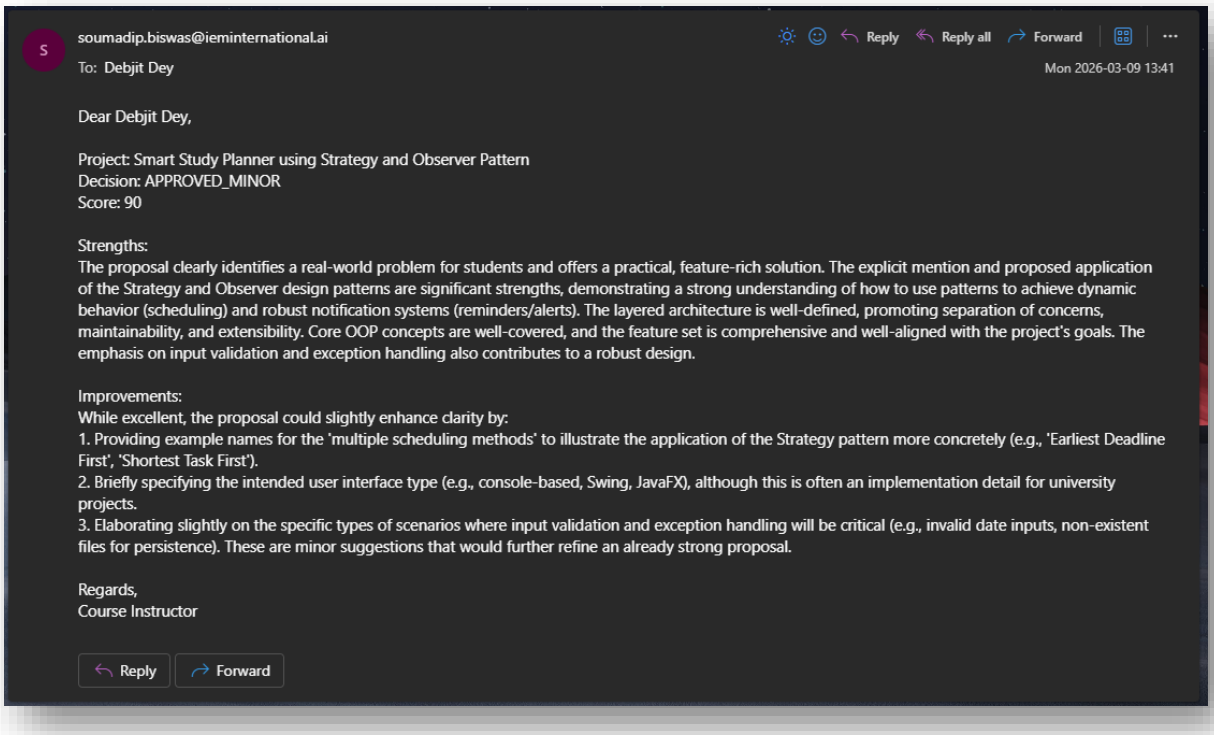
EXCEPTION HANDLING : Custom exceptions for invalid input, missing data, file errors.

COLLECTIONS :
Collections (ArrayList, PriorityQueue) for task management. Optional thread for reminders.

PROPOSED ARCHITECTURE DESCRIPTION

The system will be designed using a layered architecture, where user interaction, business logic, and data storage are separated. The design will include task management, multiple scheduling models, and deadline notifications. Modularity, extensibility, and separation of concerns are also included.

Review Mail :



S

soumadip.biswas@ieminternational.ai

Reply Reply all Forward

To: Debjit Dey

Mon 2026-03-09 13:41

Dear Debjit Dey,

Project: Smart Study Planner using Strategy and Observer Pattern
Decision: APPROVED_MINOR
Score: 90

Strengths:
The proposal clearly identifies a real-world problem for students and offers a practical, feature-rich solution. The explicit mention and proposed application of the Strategy and Observer design patterns are significant strengths, demonstrating a strong understanding of how to use patterns to achieve dynamic behavior (scheduling) and robust notification systems (reminders/alerts). The layered architecture is well-defined, promoting separation of concerns, maintainability, and extensibility. Core OOP concepts are well-covered, and the feature set is comprehensive and well-aligned with the project's goals. The emphasis on input validation and exception handling also contributes to a robust design.

Improvements:
While excellent, the proposal could slightly enhance clarity by:
1. Providing example names for the 'multiple scheduling methods' to illustrate the application of the Strategy pattern more concretely (e.g., 'Earliest Deadline First', 'Shortest Task First').
2. Briefly specifying the intended user interface type (e.g., console-based, Swing, JavaFX), although this is often an implementation detail for university projects.
3. Elaborating slightly on the specific types of scenarios where input validation and exception handling will be critical (e.g., invalid date inputs, non-existent files for persistence). These are minor suggestions that would further refine an already strong proposal.

Regards,
Course Instructor

Reply Forward